

✓ Lab 10 : Implementation of Graph Neural Network for Link Prediction

```
# Installing some libraires :
%%capture
import torch
version = torch.__version__
i = version.find('+')
version = version[:i-1] + '0' + version[i:]
url = 'https://data.pyg.org/whl/torch-' + version + '.html'
!pip install torch-scatter -f $url
!pip install torch-sparse -f $url
!pip install torch-geometric
!pip install torch-cluster -f $url
!pip install pygdn
!pip install --upgrade scipy

# Importing Necessary Libraries :
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import torch
from torch_geometric.datasets import Amazon
import torch_geometric.transforms as T
from torch_geometric.nn import GCNConv
from torch_geometric.utils import negative_sampling
from sklearn.metrics import roc_auc_score

import warnings
warnings.filterwarnings('ignore')

# Loading the AMAZON dataset :
dataset = Amazon(root='./data', name='Computers') # or 'Photo'
graph = dataset[0]

📄 Downloading https://github.com/shchur/gnn-benchmark/raw/master/data/npz/amazon\_electronics\_computers.npz
Processing...
Done!

# Deleting the node classification masks :
del graph.train_mask
del graph.val_mask
del graph.test_mask

# Splitting the data :
split = T.RandomLinkSplit(
    num_val=0.05,
    num_test=0.1,
    is_undirected=True,
    add_negative_train_samples=False,
    neg_sampling_ratio=1.0,
)

train_data, val_data, test_data = split(graph)

print("train_data:", train_data)
print("val_data:", val_data)
print("test_data:", test_data)

📄 train_data: Data(x=[13752, 767], edge_index=[2, 417964], y=[13752], edge_label=[208982], edge_label_index=[2, 208982])
val_data: Data(x=[13752, 767], edge_index=[2, 417964], y=[13752], edge_label=[24586], edge_label_index=[2, 24586])
test_data: Data(x=[13752, 767], edge_index=[2, 442550], y=[13752], edge_label=[49172], edge_label_index=[2, 49172])

# Link Prediction Model :
class Net(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super().__init__()
        self.conv1 = GCNConv(in_channels, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, out_channels)

    def encode(self, x, edge_index):
        x = self.conv1(x, edge_index).relu()
        return self.conv2(x, edge_index)

    def decode(self, z, edge_label_index):
```

```

        return (z[edge_label_index[0]] * z[edge_label_index[1]]).sum(dim=-1)

def decode_all(self, z):
    prob_adj = z @ z.t()
    return (prob_adj > 0).nonzero(as_tuple=False).t()

# Training and Evaluation :
def train_link_predictor(model, train_data, val_data, optimizer, criterion, n_epochs=100):
    for epoch in range(1, n_epochs + 1):
        model.train()
        optimizer.zero_grad()
        z = model.encode(train_data.x, train_data.edge_index)

        # Negative sampling (same number of negative samples as positives)
        neg_edge_index = negative_sampling(
            edge_index=train_data.edge_index,
            num_nodes=train_data.num_nodes,
            num_neg_samples=train_data.edge_label_index.size(1),
            method='sparse'
        )

        edge_label_index = torch.cat(
            [train_data.edge_label_index, neg_edge_index],
            dim=-1,
        )
        edge_label = torch.cat([
            train_data.edge_label,
            train_data.edge_label.new_zeros(neg_edge_index.size(1))
        ], dim=0)

        out = model.decode(z, edge_label_index).view(-1)
        loss = criterion(out, edge_label.float())
        loss.backward()
        optimizer.step()

        val_auc = eval_link_predictor(model, val_data)

        if epoch % 10 == 0:
            print(f"Epoch: {epoch:03d}, Train Loss: {loss:.3f}, Val AUC: {val_auc:.3f}")

    return model

@torch.no_grad()
def eval_link_predictor(model, data):
    model.eval()
    z = model.encode(data.x, data.edge_index)
    out = model.decode(z, data.edge_label_index).view(-1).sigmoid()
    return roc_auc_score(data.edge_label.cpu().numpy(), out.cpu().numpy())

# Running the training :
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = Net(dataset.num_features, 128, 64).to(device)
train_data = train_data.to(device)
val_data = val_data.to(device)
test_data = test_data.to(device)

optimizer = torch.optim.Adam(params=model.parameters(), lr=0.01)
criterion = torch.nn.BCEWithLogitsLoss()

model = train_link_predictor(model, train_data, val_data, optimizer, criterion)

➡ Epoch: 010, Train Loss: 0.693, Val AUC: 0.500
Epoch: 020, Train Loss: 0.694, Val AUC: 0.500
Epoch: 030, Train Loss: 0.695, Val AUC: 0.500
Epoch: 040, Train Loss: 0.695, Val AUC: 0.500
Epoch: 050, Train Loss: 0.695, Val AUC: 0.500
Epoch: 060, Train Loss: 0.695, Val AUC: 0.500
Epoch: 070, Train Loss: 0.695, Val AUC: 0.500
Epoch: 080, Train Loss: 0.694, Val AUC: 0.500
Epoch: 090, Train Loss: 0.694, Val AUC: 0.500
Epoch: 100, Train Loss: 0.694, Val AUC: 0.500

# Final Evaluation using AUC (Area Under the Curve) :
test_auc = eval_link_predictor(model, test_data)
print(f"\nTest AUC: {test_auc:.3f}")

```



Test AUC: 0.500

